

Restful API — 复习要点

一、前后端分离的基本概念

1.1 第一阶段：JSP/PHP 时代

[p.5]

- **优点：**开发简单（一个 JSP 文件搞定）；部署方便；SEO 友好（服务端渲染, Server-Side Rendering）；首屏速度快。
- **缺点：**职责混乱（Java + HTML + CSS + JS 混在一起）；难以维护（牵一发而动全身）；前端无法独立开发（依赖后端环境）；无法复用（每个页面都要重新渲染）。

1.2 第二阶段：MVC (Model-View-Controller) 框架时代

[p.6]

- **优点：**职责分离（Controller – Service – Dao）；可测试性提升（可以单元测试, Unit Test）；代码复用（Service 层可被多个 Controller 调用）。
- **缺点：**页面跳转由后端控制；静态资源混合部署（性能瓶颈）；无法适应移动端（一套模板打天下）；前端仍然依赖后端。

1.3 第三阶段：前后端分离时代

[p.7–11]

为什么需要前后端分离？

[p.7]

- **传统单体架构 (Monolithic Architecture) 的局限：**渲染页面、业务逻辑和数据访问都在同一代码库中运行，系统复杂度增加后扩展困难、上线周期长、前端无法独立迭代。
- **不同客户端需求的冲突：**桌面端 (Desktop) 和移动端 (Mobile) 需要单一后端频繁兼顾不同终端的接口需求，造成开发瓶颈和协调成本。

什么是前后端分离？

[p.8]

- **核心理念：**把用户界面层（前端, UI Layer）和业务数据层（后端, Business/Data Layer）解耦 (Decouple)，双方通过统一的 API 通信。
- **前端代码与后端代码在不同的服务器实例上运行，**后端只暴露数据接口，不再负责渲染页面。前端通常通过 RESTful API 调用后端获取数据。

- 这种架构也被称为 “headless”（无头）或 “decoupled”（解耦）架构。

前后端分离的优势

[p.9]

- **灵活的技术栈和独立部署：**前端可选 Vue、React 等框架；后端可选适合的语言和框架；两个服务独立部署，互不影响。
- **缩短迭代周期、提升协作效率：**前端可通过模拟接口 (Mock API) 并行开发；后端变更只需保证 API 契约 (API Contract) 不变，前端不受影响。
- **面向多端的可扩展性：**通过 BFF (Backend-for-Frontend) 层为不同终端（桌面端、移动端）定制接口，实现性能优化和体验差异化。
- **更容易实施微服务 (Microservices) 和云原生 (Cloud Native)：**前后端分离为微服务拆分提供天然接口界限，方便通过 API 网关 (API Gateway) 统一治理。

二、Web 服务 (Web Service) 的实现方式

2.1 Web Service 简介

[p.13]

- **Web Service (Web 服务)** 是一种基于 Web 的远程调用 (Remote Call) 技术，主要用于不同系统之间的互操作 (Interoperability)。
- **核心理念：**实现 “基于 Web 无缝集成”；实现不同系统间的相互调用（语言无关, Language-independent、平台无关, Platform-independent）——Anywhere、Anytime、AnyDevice。
- **多种实现方式：**SOAP (Simple Object Access Protocol, 简单对象访问协议) 和 REST (Representational State Transfer, 表述性状态转移)。

2.2 基于 SOAP 的 Web Service

[p.15–18]

- **基于 SOAP 和 XML：**传统 Web Service 主要使用 SOAP 协议，依赖 XML 进行数据传输。
- **WSDL (Web Service Description Language, Web 服务描述语言)：**提供服务描述，定义可用的方

法和数据格式，通常用于生成客户端代码。

- UDDI (Universal Description, Discovery, and Integration, 统一描述、发现和集成协议)。
- 协议支持: 支持多种传输协议, 如 HTTP、HTTPS、SMTP、JMS。

2.3 REST 替代 SOAP 的原因

[p.20-21]

REST 在大部分应用场景中更受欢迎: 轻量级、高性能、JSON 数据格式简洁易读、适合移动端和微服务架构。

2.4 SOAP Web Service 对比 RESTful API

[p.21]

协议 (Protocol)	SOAP 支持 HTTP、SMTP、JMS 等; REST 主要基于 HTTP
数据格式 (Data Format)	SOAP 主要是 XML; REST 使用 JSON、XML (JSON 更常见)
消息结构 (Message)	SOAP 复杂 (SOAP Envelope); REST 简单 (HTTP + JSON)
状态管理 (State)	SOAP 可以有状态 (Stateful) 或无状态 (Stateless); REST 无状态
性能 (Performance)	SOAP 因 XML 解析较慢、性能较低; REST 轻量级、性能更好
安全性 (Security)	SOAP 内置 WS-Security, 支持 SOAP 认证、加密等; REST 需手动实现 (如 OAuth 2.0、JWT)
适用场景 (Use Case)	SOAP: 企业级应用、银行、金融、政府等高安全性和事务性 (Transaction) 场景; REST: Web 应用、微服务、移动应用、物联网 (IoT) 等

三、REST 简介

3.1 什么是 REST?

[p.22-23]

- REST (Representational State Transfer, 表述性状态转移/表现层状态转移) 由 Roy Thomas Fielding 在 2000 年其博士论文中提出。
- Roy Fielding 是 HTTP 规范的主要作者之一, Apache HTTP Server 项目的联合创始人, 计算机体系结构领域的权威。
- REST 是符合 Web Service 思想的一种架构设计风格 (Architectural Style), 不是标准 (Stan-

dard)。

- 使用 HTTP、URL、XML、JSON、HTML 等广泛流行的标准和协议。
- 设计理念: 轻量级 (Lightweight)、跨平台 (Cross-platform)、跨语言 (Cross-language)。
- 一个符合 REST 设计原则的架构就可以称为 RESTful 架构。

3.2 REST 的六大约束 (Six Constraints)

[p.24]

1. **客户端-服务器 (Client-Server):** 关注点分离 (Separation of Concerns), 客户端负责用户界面, 服务器负责数据存储, 二者可独立演化。
2. **无状态 (Stateless):** 每个请求必须包含所有必要信息, 服务器不保存客户端上下文 (Context)。
3. **可缓存 (Cacheable):** 响应应显式标记是否可缓存, 减少客户端与服务器之间的交互, 提升性能。
4. **统一接口 (Uniform Interface):** 资源标识 (URI)、通过表述操作资源、自描述消息、HATEOAS (超媒体作为应用状态引擎, Hypermedia As The Engine Of Application State)。
5. **分层系统 (Layered System):** 客户端无法辨别连接的是终端服务器还是中间层, 支持负载均衡 (Load Balancing) 和缓存。
6. **按需代码 (Code on Demand, 可选):** 服务器可临时向客户端传输可执行代码 (如 JavaScript), 扩展客户端功能。

3.3 REST 与 RESTful API 的关系

[p.25]

- 如果一个 API 完全遵循上述六大约束或原则, 就被称为 RESTful API。
- 实际应用中, 很多 API 并不完全符合 REST 的所有约束, 仍然称为 RESTful API。
- 实际项目应用中总结出的一些常用使用方式——最佳实践 (Best Practices)。

四、RESTful API 最佳实践

4.1 请求设计 (Request Design)

[p.26]

- **URI 使用名词 (Noun), 尽量使用复数, 如 /orders。**

- **URI 使用嵌套 (Nested)** 表示关联关系，如 `/users/1/orders/3` (用户 1 的订单 3)。
- 使用正确的 HTTP 方法 (HTTP Methods): GET、POST、PUT、PATCH、DELETE 等。
- 不符合 CRUD 的操作，使用以下三种方式之一：
 - (1) POST + 动词 (如 `POST /orders/1/cancel`);
 - (2) 查询字符串中带 action 字段 (如 `POST /orders/1?action=cancel`);
 - (3) 设计子资源 (Sub-resource) (如 `POST /orders/1/cancellation`)。
- 网络协议建议使用 **HTTPS** 协议，而不是 HTTP 协议。
- 建议使用标准的身份认证和鉴权规范，如 JWT (JSON Web Token)、OAuth 框架等。
- 在响应内容中，最好包含相关其他 API 的**链接 (Links)** (HATEOAS 原则)。

4.4 统一响应格式范例

[p.34–35]

典型统一响应结构 (JSON 格式):

```
{
  "code": 200,
  "message": "success",
  "data": { ... }
}
```

- 统一格式便于客户端统一处理成功和错误两种情况。
- 错误时 data 可为 null，message 中包含具体错误信息。

4.5 常用 HTTP 状态码

[p.31, p.34–35]

- **2xx 成功**: 200 OK (请求成功)、201 Created (资源创建成功, POST 返回)、204 No Content (删除成功, 无返回内容)。
- **3xx 重定向**: 301 Moved Permanently (永久重定向)、304 Not Modified (资源未修改, 缓存相关)。
- **4xx 客户端错误**: 400 Bad Request (请求参数有误)、401 Unauthorized (未认证, 需登录)、403 Forbidden (已认证但无权限, 鉴权失败)、404 Not Found (资源不存在)、405 Method Not Allowed (HTTP 方法不允许)、409 Conflict (资源冲突)。
- **5xx 服务端错误**: 500 Internal Server Error (服务器内部错误)、502 Bad Gateway (网关错误)、503 Service Unavailable (服务不可用)。

4.2 HTTP 请求方法 (HTTP Verbs / Methods)

[p.27]

- **GET (SELECT)**: 从服务器获取资源 (一项或多项), 安全且幂等 (Idempotent)。
- **POST (CREATE)**: 在服务器新建一个资源, 非幂等。
- **PUT (UPDATE)**: 更新资源 (客户端提供改变后的**完整**资源), 幂等。
- **PATCH (UPDATE)**: 部分更新资源 (客户端提供需改变的部分资源), 非幂等。
- **DELETE (DELETE)**: 删除资源, 幂等。
- **HEAD**: 获取资源头部 (元数据, Metadata), 与 GET 类似但不返回响应体。
- **OPTIONS**: 获取资源信息, 如某资源有哪些接口 (HTTP 方法) 可以使用。

CRUD 与 HTTP 方法对应

[p.29]

- Create (创建) → POST
- Read (读取) → GET
- Update (更新) → PUT / PATCH
- Delete (删除) → DELETE

购物车复杂场景范例

[p.30]

- 复杂操作 (如加入购物车、下单、取消订单等) 不直接映射到标准 CRUD 操作时, 使用 POST + 动词方式或子资源方式处理。

4.3 响应设计 (Response Design)

[p.31]

- 响应内容中应包含详细且正确的 HTTP 状态码 (HTTP Status Code)。
- 响应内容采用**统一响应格式 (Unified Response Format)**, 返回数据使用 JSON。

五、Spring MVC REST 相关注解

[p.37–39]

- `@RestController` —— 类级别, 组合了 `@Controller` 和 `@ResponseBody`, 所有方法返回值自

动序列化为 JSON 写入 HTTP 响应体。

- **@RequestMapping** —— 类/方法级别，映射 HTTP 请求到控制器方法；可指定 path（或 value）、method、params、headers 等属性。
- **@GetMapping / @PostMapping / @PutMapping / @PatchMapping / @DeleteMapping** —— 方法级别快捷注解，分别对应各自 HTTP 方法。
- **@PathVariable** —— 参数级别，绑定 URL 路径中的模板变量（如 /users/{id} 中的 id）。
- **@RequestParam** —— 参数级别，绑定 URL 查询参数（Query Parameter）（如 ?page=1&size=10）。
- **@RequestBody** —— 参数级别，将 HTTP 请求体（Request Body）中的 JSON 反序列化（Deserialize）为 Java 对象。
- **@ResponseBody** —— 方法级别，将返回值序列化（Serialize）写入 HTTP 响应体（@RestController 已包含此功能）。
- **@ResponseStatus** —— 方法级别，指定返回的 HTTP 状态码（如 @ResponseStatus(HttpStatus.CREATED)）。
- **@CrossOrigin** —— 类/方法级别，解决跨域请求（CORS）问题。
- **@ExceptionHandler** —— 方法级别，处理特定异常并返回自定义响应。
- **@ControllerAdvice** —— 类级别，全局异常处理和全局数据绑定。

六、跨域问题—CORS (Cross-Origin Resource Sharing)

6.1 跨域问题的来源

[p.40]

- 跨域问题的由来是浏览器的**同源策略 (Same-Origin Policy)**。
- **同源策略**：浏览器的安全策略之一，指一个网页的内容只能来自于同一个服务器来源。
- **同源 (Same Origin) 判断标准**：两个 URL 如果协议 (Protocol)、主机名 (Host)、端口 (Port) 均一致则同源，三者中有一个不相同即不同源（跨域，Cross-Origin）。
- 示例：对于基准 URL `http://www.csu.edu.cn/cs/index.html`：
 - `http://www.csu.edu.cn/cs1/page.html` —— 同源（只有路径不同）

- `https://www.csu.edu.cn/cs/page.html` —— 不同源（协议不同，HTTP vs HTTPS）
- `http://www.csu.edu.cn:8080/cs/page.html` —— 不同源（端口不同，80 vs 8080）
- `http://mail.csu.edu.cn/cs1/page.html` —— 不同源（主机不同，www vs mail）

6.2 Spring Boot 中解决跨域问题的三种方式

[p.41]

- **全局配置 (Global Configuration)**：通过配置类实现 `WebMvcConfigurer` 接口，重写 `addCorsMappings` 方法全局配置 CORS。
- **局部配置 (Local Configuration)**：使用 `@CrossOrigin` 注解局部配置某个 Controller 或方法。
- **过滤器方式 (Filter)**：通过 `CorsFilter` 针对某些 URL 进行跨域配置。

七、用户认证 (Authentication)

7.1 传统架构中的会话跟踪 (Session Tracking)

[p.43]

- **前后端未分离架构**：前端登录 → 后端生成 `jsessionid` 并保存到 Session 中（连同用户 ID）→ 将 `jsessionid` 传给用户存入浏览器 Cookie → 后续请求带上 Cookie → 后端根据 Cookie 值查询用户、验证是否过期。
- **前后端分离后的问题**：
 - 问题 1：前后端分离后浏览器不能自动使用 Cookie、存在跨域问题（Cookie 跨域限制）、或根本没有浏览器可用（如移动端 App、IoT 设备）。
 - 问题 2：服务器进行系统架构后（分布式/集群环境，Distributed/Clustered），Session 不便于存储和共享。

7.2 Token 机制 (Token-based Authentication)

[p.44]

- **Token（令牌）**在 Web 应用中特指访问资源的凭据（Credential）。
- 用户登录成功后，服务器基于某种机制生成一个字符串——token。
- **Token 机制与 Session 机制的核心区别是**：服

务器是否存储。Session 在服务器端存储（有状态, Stateful）；Token 通常是无状态的 (Stateless)，服务器不存储，Token 本身自包含用户信息。

- 之后的请求客户端都带上 token，服务器根据 token 进行用户的认证。
- Token 机制需解决 token 的加密 (Encryption)、解密 (Decryption)、过期 (Expiration) 等问题。
- 广义上：SessionId 也是 token 机制的一种实现；Cookie 也可被认为是 token 机制的一种实现。

7.3 JWT —JSON Web Token

[p.45–49]

(a) JWT 概述

[p.45]

- JWT 是目前流行的用户认证 token 解决方案，基于开放标准 **RFC 7519**。
- 定义了一种简洁的、自包含的 (Self-contained)、以 JSON 对象形式安全传递信息的方法。
- 采用**数字签名 (Digital Signature)**，可以被验证和信任。
- 可选加密（使用 HMAC 算法密钥、RSA 或 ECDSA 公/私钥机制）。
- Spring Security 组件对 JWT 提供集成支持。

(b) JWT 的结构 (三部分)

[p.46–47]

- JWT 的 token 字符串由三部分组成，每部分均为一个 JSON 对象，用 Base64 URL 安全编码后以句点 (.) 分隔：**Header.Payload.Signature**
- **Header (头部)** [p.47]：描述 JWT 元数据。包含：
 - alg: 签名算法（如 HS256、RS256）
 - typ: token 类型 (JWT)
- **Payload (载荷/负载)** [p.47]：存放实际需传递的数据，包含标准字段 (Registered Claims) 和自定义字段 (Private Claims)。官方标准字段：
 - iss (Issuer, 签发人) exp (Expiration Time, 过期时间)
 - sub (Subject, 主题) aud (Audience, 使用者/受众)
 - nbf (Not Before, 生效时间) iat (Issued At, 签发时间)
 - jti (JWT ID, 唯一编号)

- **Signature (签名)** [p.47]：对 Header 和 Payload 的签名，防止数据篡改。默认使用 HMAC SHA256：
`HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret)`

(c) JWT 的特点

[p.48]

- JWT 默认不加密，但可选加密算法进行加密，可在生成原始 Token 后再加密一次。
- JWT 不仅可用于认证，也可用于数据传输（在自定义字段中传输数据，减少对服务器的访问请求）。
- **主要缺点**：token 只要发出，无法在使用过程中废止或修改 token（没有内置的撤销机制）。
- 因 JWT 包含认证信息，为防止泄漏应设置尽可能短的有效时间（即通过 exp 字段控制）。
- JWT 建议采用 **HTTPS** 协议传输，尽量不使用 HTTP 协议。

(d) JWT 的使用过程

[p.49]

- **首次请求 (登录阶段)：**

- 步骤 1. 客户端发送账号和密码（登录请求）。
- 步骤 2. 服务器验证成功，生成 JWT 令牌 (Token) 并返回给客户端。
- 步骤 3. 客户端保存令牌（如存储在 localStorage 或内存中）。

- **后续请求 (认证阶段)：**

- 步骤 1. 客户端在请求头中携带 JWT 令牌（通常格式：**Authorization: Bearer <token>**）。
- 步骤 2. 服务器验证令牌（检查签名是否正确、令牌是否过期等）。
- 步骤 3. 验证通过后完成响应。

八、用户鉴权 (Authorization) 与 OAuth 框架

8.1 鉴权问题来源

[p.50]

- **鉴权 (Authorization)**，也叫授权，是指用户访问 API 的权限问题——某个用户只能访问某些 API，而不能访问其他 API。
- 鉴权常用于系统的**第三方登录**的应用场景。
- **经典类比——快递员送外卖**：如何让快递员能自由进入小区送货，又不必知道业主密码，且唯

一权限就是送货（其他需要密码的场合没有权限）？该案例说明了**认证 (Authentication)** 与 **鉴权 (Authorization)** 的分离需求。

8.2 鉴权机制的设计（快递员范例）

[p.51]

- 第 1 步：门禁系统的密码输入器下面增加一个“获取授权”按钮，快递员首先按此按钮申请授权。
- 第 2 步：快递员按下按钮后，业主得知有人正在要求授权，系统显示快递员信息（姓名、工号、所属快递公司），业主确认后通知门禁系统放行。
- 第 3 步：门禁系统得到确认后，向快递员颁发一个进入小区的令牌 (Access Token)，只在短期内（比如七天）有效。
- 第 4 步：快递员向门禁系统输入令牌，进入小区。

该范例对应 OAuth 授权码模式 (Authorization Code) 的核心思想：第三方应用（快递员）获得有限的、临时的访问权限 (Access Token)，而无需用户的完整凭证（密码）。

8.3 OAuth 2.0 鉴权框架

[p.52]

- OAuth 是一个鉴权 (Authorization) 的**开放标准**，在软件开发中广泛使用，目前最新版本是 2.0 (<https://oauth.net/>)。
- OAuth 为桌面、手机或 Web 应用提供了一种简单的、标准的方式去访问需要用户授权的 API 服务。
- Spring Framework 中提供 **Spring Security OAuth2** 组件，实现了 OAuth 规范。
- **核心概念区分：**
 - **认证 (Authentication)：**验证“你是谁” (Who are you?)，如 JWT Token 确认用户身份。
 - **鉴权/授权 (Authorization)：**验证“你能做

什么” (What can you do?)，如 OAuth 2.0 授予第三方有限访问权限。

九、本章小结

[p.53]

三大核心主题：

1. **前后端分离的基本概念：**JSP/PHP 时代 → MVC 时代 → 前后端分离时代；前后端分离的定义（解耦、通过 API 通信）；优势（独立部署、并行开发、BFF 层、微服务就绪）。
2. **Web 服务的实现方式：**基于 SOAP 的 Web Service (XML + WSDL + UDDI、多协议支持、企业级) vs 基于 REST 的 RESTful API (HTTP + JSON、轻量级、无状态、Web/移动/物联网)。
3. **Spring Boot RESTful API 实现：**核心注解 (@RestController、@RequestMapping 及各快捷注解)；CORS 跨域三种解决方案（全局配置 / @CrossOrigin / 过滤器）；Token / JWT 用户认证 (Header.Payload.Signature 三部分结构、无状态、HTTPS 传输、短有效期)；OAuth 2.0 用户鉴权 (Access Token、授权码模式)。

RESTful API 设计要点速记：

- **URI:** 名词、复数、嵌套表示关联 (/users/1/orders/3)
- **HTTP 方法：**GET (查) / POST (增) / PUT (全量改) / PATCH (部分改) / DELETE (删)
- **响应 (Response)：**统一格式 (code + message + data)、JSON、HTTPS、正确状态码、含链接 (HATEOAS)
- **认证 (Authentication)：**JWT (Header.Payload.Signature 三部分结构、无状态、HTTPS 传输、短有效期)
- **鉴权 (Authorization)：**OAuth 2.0、Access Token
- **安全：**CORS 同源策略 (协议 + 主机 + 端口)、@CrossOrigin、全局/局部/过滤器三种方案